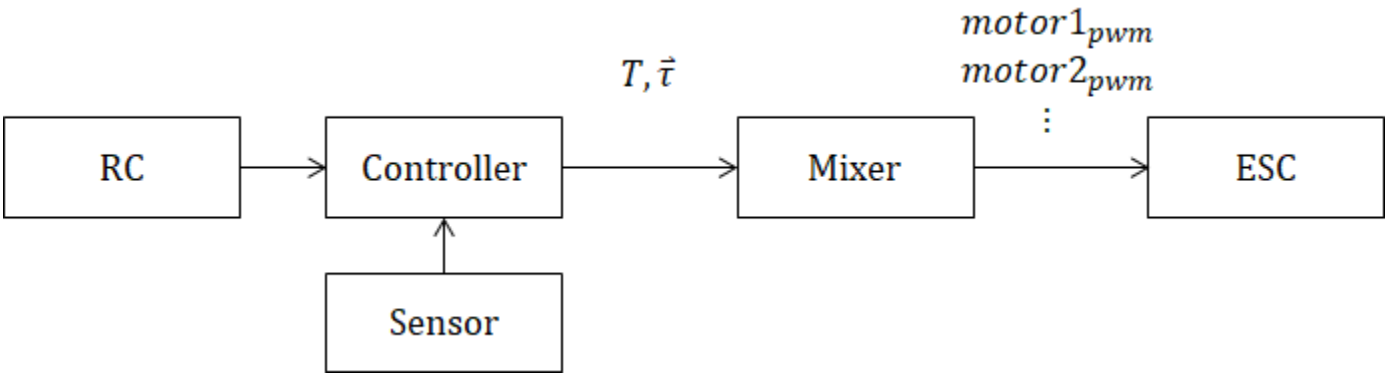
Motor Mixer Theory

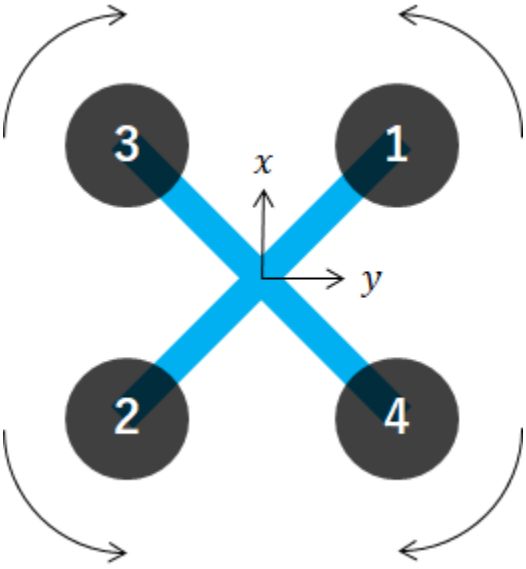
This page describes the theory behind motor mixing for multirotors. The reader will be familiar with how to come up with mixer tables for any motor configuration. Pseudo-inverse technique is used.

Overview

A typical control architecture for multirotor vehicles is as follows:



The controller outputs desired thrust, T, and roll/pitch/yaw torques, τ. Then the mixer converts those values to individual motor commands. For example, let an X-configuration quadrotor of the following form:



Arrow indicates the direction of rotation of the propeller when looking down at the quadcopter from above. Note that the clockwise propeller causes a counter-clockwise moment on the quadcopter. The mixer algorithm for this drone is as follows:

```
motor1_pwm = thrust - roll + pitch + yaw
```

```
motor2_pwm = thrust + roll - pitch + yaw
motor3_pwm = thrust + roll + pitch - yaw
motor4_pwm = thrust - roll - pitch - yaw
```

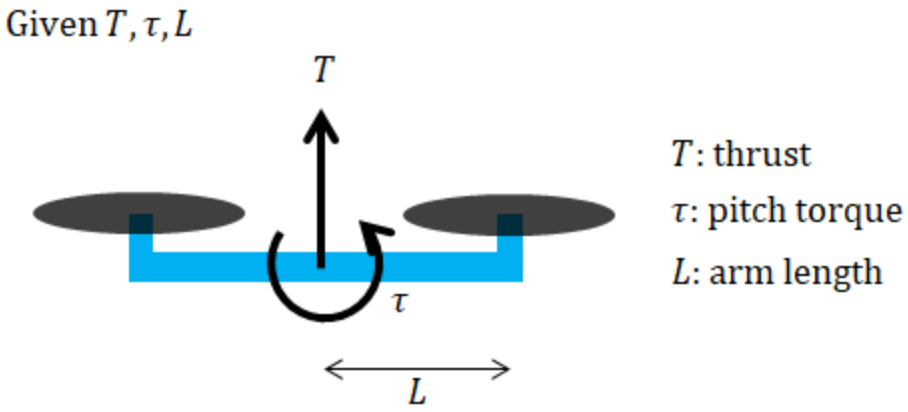
We are using front-right-down (roll-pitch-yaw) coordinates and thrust is in up direction. The above math can be rewritten in matrix form as,

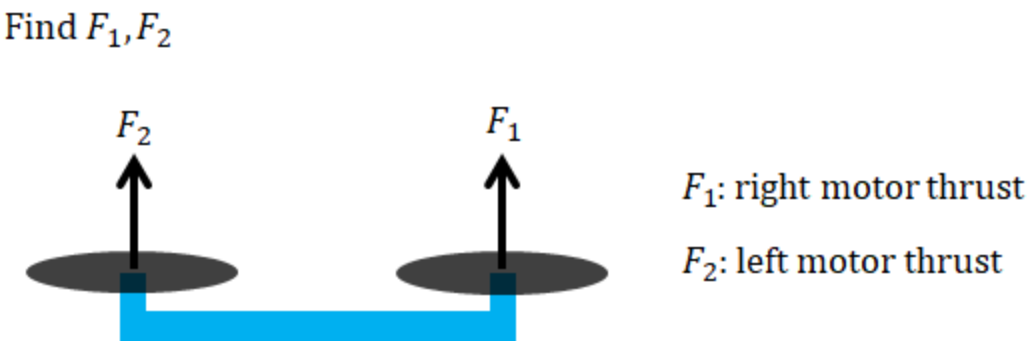
$$\begin{bmatrix} motor1_{pwm} \\ motor2_{pwm} \\ motor3_{pwm} \\ motor4_{pwm} \end{bmatrix} = \begin{bmatrix} 1 & -1 & 1 & 1 \\ 1 & 1 & -1 & 1 \\ 1 & 1 & 1 & -1 \\ 1 & -1 & -1 & -1 \end{bmatrix} \begin{bmatrix} thrust \\ roll \\ pitch \\ yaw \end{bmatrix}$$

The matrix in the middle with 1s and -1s is called a mixer table. It is an N-by-4 matrix where N is the number of rotors. It is possible to guess this matrix just by intuition but it soon becomes complicated when there are more than 4 motors. The following sections explain a method to mathematically determine this matrix. This problem is part of a broader field called "control allocation".

2D Case

To simplify things, let us first consider the 2D case. Given T (thrust) and τ (pitch torque) and L (arm length), the problem is to determine F_1 and F_2 (forces from the right and left motors respectively).





Summing the forces and torques, we get:

$$T = F_1 + F_2$$
$$\tau = LF_1 - LF_2$$

In matrix form:

$$\begin{bmatrix} T \\ \tau \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ L & -L \end{bmatrix} \begin{bmatrix} F_1 \\ F_2 \end{bmatrix}$$

Solve for the individual motor thrusts by inverting the matrix:

$$\begin{bmatrix} F_1 \\ F_2 \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ L & -L \end{bmatrix}^{-1} \begin{bmatrix} T \\ \tau \end{bmatrix}$$
$$= \begin{bmatrix} 1/2 & L/2 \\ 1/2 & -L/2 \end{bmatrix} \begin{bmatrix} T \\ \tau \end{bmatrix}$$

This can be normalized by separating the matrix:

$$\begin{bmatrix} F_1 \\ F_2 \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} 1/2 & 0 \\ 0 & L/2 \end{bmatrix} \begin{bmatrix} T \\ \tau \end{bmatrix}$$

$$\begin{bmatrix} F_1 \\ F_2 \end{bmatrix}_{normalized} = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} T \\ \tau \end{bmatrix}$$

So the motor mixer table is given by,

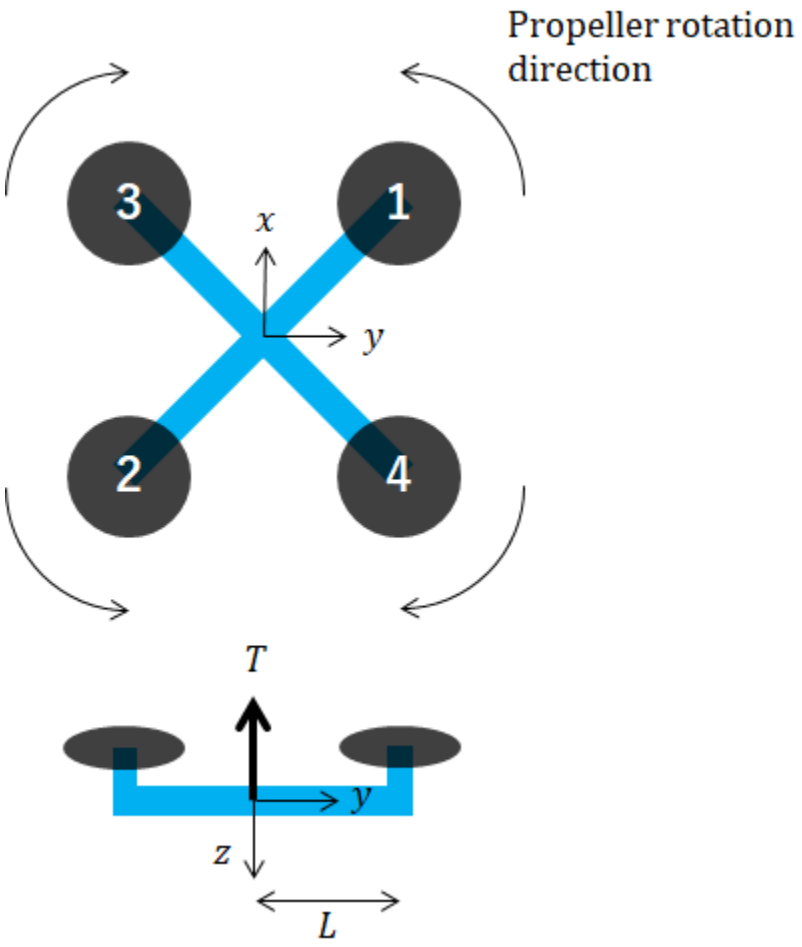
$$\begin{matrix} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} & \begin{matrix} \text{motor 1} \\ \text{motor 2} \end{matrix} \\ \begin{matrix} \text{thrust} & \text{pitch} \end{matrix} & \end{matrix}$$

So the final algorithm is,

```
motor1_pwm = thrust + pitch
motor2_pwm = thrust - pitch
```

3D Case

Now let us consider a 3D case. We will use X-configuration quadrotor again as we already know the answer.



Summing the forces and torques, we get:

$$
\begin{aligned}
T &= F_1 + F_2 + F_3 + F_4 \\
\tau_x &= -LF_1 + LF_2 + LF_3 - LF_4 \\
\tau_y &= LF_1 - LF_2 + LF_3 - LF_4 \\
\tau_z &= \gamma F_1 + \gamma F_2 - \gamma F_3 - \gamma F_4
\end{aligned}
$$

γ is rotor drag to thrust coefficient given by,

$$M = k_M n^2$$

$$F = k_T n^2$$

$$\frac{M}{F} = \frac{k_M}{k_T} = \gamma$$

where n is rotor rotation speed, k_T is thrust coefficient, and k_M is moment coefficient. k_T and k_M are only dependent on rotor geometry (and so is γ).

In matrix form,

$$\begin{bmatrix} T \\ \tau_x \\ \tau_y \\ \tau_z \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ -L & L & L & -L \\ L & -L & L & -L \\ \gamma & \gamma & -\gamma & -\gamma \end{bmatrix} \begin{bmatrix} F_1 \\ F_2 \\ F_3 \\ F_4 \end{bmatrix}$$

Solve for the individual motor thrusts by inverting the matrix

$$\begin{bmatrix} F_1 \\ F_2 \\ F_3 \\ F_4 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ -L & L & L & -L \\ L & -L & L & -L \\ \gamma & \gamma & -\gamma & -\gamma \end{bmatrix}^{-1} \begin{bmatrix} T \\ \tau_x \\ \tau_y \\ \tau_z \end{bmatrix}$$

$$= \left(\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & L & 0 & 0 \\ 0 & 0 & L & 0 \\ 0 & 0 & 0 & \gamma \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 & 1 \\ -1 & 1 & 1 & -1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \end{bmatrix} \right)^{-1} \begin{bmatrix} T \\ \tau_x \\ \tau_y \\ \tau_z \end{bmatrix}$$

$$= \begin{bmatrix} 1 & -1 & 1 & 1 \\ 1 & 1 & -1 & 1 \\ 1 & 1 & 1 & -1 \\ 1 & -1 & -1 & -1 \end{bmatrix} \begin{bmatrix} 1/4 & 0 & 0 & 0 \\ 0 & 1/4 & 0 & 0 \\ 0 & 0 & 1/4 & 0 \\ 0 & 0 & 0 & 1/4 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1/L & 0 & 0 \\ 0 & 0 & 1/L & 0 \\ 0 & 0 & 0 & 1/\gamma \end{bmatrix} \begin{bmatrix} T \\ \tau_x \\ \tau_y \\ \tau_z \end{bmatrix}$$

The normalized mixer table is:

$$\begin{bmatrix} 1 & -1 & 1 & 1 \\ 1 & 1 & -1 & 1 \\ 1 & 1 & 1 & -1 \\ 1 & -1 & -1 & -1 \end{bmatrix} \begin{matrix} \text{motor 1} \\ \text{motor 2} \\ \text{motor 3} \\ \text{motor 4} \end{matrix}$$

thrust roll pitch yaw

A General Solution

Given N rotors positioned at (x_1, y_1), (x_2, y_2), ... (x_N, y_N), the general from is:

$$\begin{bmatrix} T \\ \tau_x \\ \tau_y \\ \tau_z \end{bmatrix} = \begin{bmatrix} 1 & 1 & \dots & 1 \\ -y_1 & -y_1 & \dots & -y_N \\ x_1 & x_2 & \dots & x_N \\ \pm\gamma & \pm\gamma & \dots & \pm\gamma \end{bmatrix} \begin{bmatrix} F_1 \\ F_2 \\ \vdots \\ F_N \end{bmatrix}$$

$$\vec{W} = \qquad A \qquad \vec{u}$$

where $+\gamma$ is for propeller in CCW rotation and $-\gamma$ for propeller in CW rotation. The solution to the above equation is:

$$\vec{u} = A^{+}\vec{W}$$

where $^{+}$ is Moore-Penrose pseudo-inverse. A python script to solve the above is as follows:

```
import numpy as np

# XYZ are in front-right-down coordinates
# rotors = [
#     Position Prop-Direction
#    [ x1, y1, 'CW'/'CCW'],
#    [ x2, y2, 'CW'/'CCW'],
#    [ x3, y3, 'CW'/'CCW'],
#    ...
# ]

# Quadrotor-X
#    3   1
#      x
#    2   4
rotors = [
    [ 1,  1, 'CCW'],
    [-1, -1, 'CCW'],
    [ 1, -1, 'CW' ],
    [-1,  1, 'CW' ]
]

def normalize(B):
    scale = np.abs(B).max(axis=0)

    # Same scale on roll and pitch
    scale[1] = max(scale[1], scale[2])
    scale[2] = scale[1]

    return B / scale

gamma = 1 # Drag to thrust coefficient

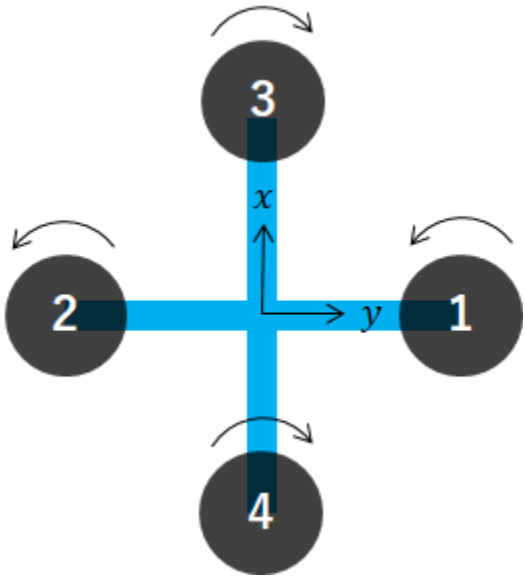
# 4 x N matrix where N is the number of rotors
# Row 1: thrust
# Row 2: roll torque (x)
# Row 3: pitch torque (y)
# Row 4: yaw torque (z)
A = np.array([
    [1] * len(rotors),
    [-i[1] for i in rotors],
    [i[0] for i in rotors],
    [-gamma if i[2]=='CW' else gamma for i in rotors]
```

```
])

B = np.linalg.pinv(A)

print(A)
print(B)
print(normalize(B))
```

Example: Quadrotor +



Let, $L = 1$, $\gamma = 1$

```
rotors = [
    [ 0,  1, 'CCW'],
    [ 0, -1, 'CCW'],
    [ 1,  0, 'CW' ],
    [-1,  0, 'CW' ]
]
```

$$A = \begin{bmatrix} 1 & 1 & 1 & 1 \\ -1 & 1 & 0 & 0 \\ 0 & 0 & 1 & -1 \\ 1 & 1 & -1 & -1 \end{bmatrix}$$

$$A^{+} = \begin{bmatrix} 0.25 & -0.5 & 0 & 0.25 \\ 0.25 & 0.5 & 0 & 0.25 \\ 0.25 & 0 & 0.5 & -0.25 \\ 0.25 & 0 & -0.5 & -0.25 \end{bmatrix}$$

$$A^{+}_{normalized} = \begin{bmatrix} 1 & -1 & 0 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & -1 \\ 1 & 0 & -1 & -1 \end{bmatrix} \begin{matrix} \text{motor 1} \\ \text{motor 2} \\ \text{motor 3} \\ \text{motor 4} \end{matrix}$$

$$\begin{matrix} \text{thrust} & \text{roll} & \text{pitch} & \text{yaw} \end{matrix}$$

Final Notes

Normalization

Often times γ is not known so it is simpler to scale roll/pitch and yaw separately. In such case, the commands from the controller is no longer in valid SI-units (e.g. N and Nm).

Motor speed vs force

The output of the mixer is fed to the ESC as pwm duty. In our calculation the output of the mixer is force whereas the ESC accepts rotational speed of the rotor. The force needs to be divided by the thrust coefficient and square rooted. However this is often not done.